TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: 151287

Need help? Post on our class slack channel or email us at support@codepath.org

Unit 3 Cheatsheet

Here's a quick reference sheet for Unit 3. While not an exhaustive list, it highlights the key syntax and concepts you'll use in this unit, plus a few optional ideas that may help with problem-solving. You're still expected to know required material from earlier units.

Sections are labeled for clarity:

- ✅ In-Scope: May appear on the assessment
- 💡 Not In-Scope: Useful, but not required

🎯 Unit Goals

- [ ] Insert variables and Python expressions into strings
- [ ] Index and access subsections of a string
- [ ] Loop over strings
- [ ] Solve problems involving strings, lists, dictionaries, or a combination of the three

General Concepts ✅ In-Scope

String Methods & Syntax

Lower Method

`s.lower()` Returns `s` as a lowercase string.Try it

- Accepts no parameters
- Returns a lowercase string.

Example Usage:

```
# Example 1: Mixed case
s = 'Hello World!'
lowered = s.lower()
print(lowered) # Prints 'hello world!'

# Example 2: All uppercase
s = 'HELLO WORLD'
lowered = s.lower()
print(lowered) # Prints 'hello world'
```

Split Method

`s.split(seperator)` Splits the string into a list along whitespace or specified separator. Try it

- `s` is the string to split
- Accepts one parameter `separator`: The characters along which the string will be split. This is an *optional* parameter. By default, the character is split along any whitespace.
- Returns a list of the substrings of `s` split by the specified separator.

Example Usage:

```
# Example 1: Split along whitespace
s = 'Never gonna give you up'
split = s.split()
```

```
print(split) # Prints ['Never', 'gonna', 'give', 'you', 'up']

# Example 2: Split along specifed separator
s = 'Never-gonna-let-you-down'
split = s.split("-")
print(split) # Prints ['Never', 'gonna', 'let', 'you', 'down']
```

Join Method

`s.join(x)` Turns an iterable `x` into a string using `s` as a separator between elements in `x`. Try it

- `s` is the separator placed between items in the specified iterable
- Accepts one parameter `x`: the iterable whose items will be joined together into a string. This is a *required* parameter.
- Returns a string of the items in `x` separated by `s`

Example Usage:

```
# Example 1: Join items in a list separated by space
lst = ['Never', 'gonna', 'run', 'around', 'and', 'desert', 'you']
joined = ' '.join(lst)
print(joined) # Prints 'Never gonna run around and desert you'

# Example 2: Join items in a list separated by dash
lst = ['Never', 'gonna', 'make', 'you', 'cry']
joined = '-'.join(lst)
print(joined) # Prints 'Never-gonna-make-you-cry'
```

Strip Method

`s.strip(characters)` Removes whitespace or specified `characters` from either end of the string. Try it

- `s` is the string to remove whitespace or characters from.
- Accepts one parameter `characters`: the characters/substring to remove from either end of `s`. This is an *optional* parameter. If no value is specified, whitespace will be removed from the beginning and end of `s`.
- Returns the string `s` with whitespace or characters removed.

Example Usage:

```
# Example 1: Strip whitespace
s = '       Never gonna say goodbye       '
stripped = s.strip()
print(stripped) # Prints 'Never gonna say goodbye'

# Example 2: Strip question marks
s = '????Never gonna tell a lie and hurt you?????'
stripped = s.strip('?')
print(stripped) # Prints 'Never gonna tell a lie and hurt you'
```

Tuples

A tuple is a data type used to store multiple items in a single variable. In most cases tuples are used to group together 2-3 items. Tuples that store 3 items are sometimes referred to as a 'triple'. Tuples are commonly used to store pairs of data together or return multiple values inside a function.

Tuples are defined using round brackets.

Example usage:

```
my_tuple = ("Mario", "Luigi")
print(my_tuple) # Prints ("Mario", "Luigi")
```

Like lists and strings, we can use indices to access elements of a tuple

Example Usage:

```
my_tuple = ("Mario", "Luigi")
mario = my_tuple[0]
luigi = my_tuple[1]
print(mario) # Prints "Mario"
print(luigi) # Prints "Luigi"
```

Like strings, tuples are immutable, meaning we cannot update the contents of a tuple. If we update the contents of a tuple, we create a new tuple.

Example usage:

```
my_tuple = (10, 20)
my_tuple[0] = 30 # Results in TypeError: 'tuple' object does not support item assignment
```

Built-In Functions

Round Function

`round(number, decimal)` Returns a given number rounded to the specified decimal. Try it

- Accepts two parameters:
  - `number`: the number to be rounded. This is a *required* argument.
  - `decimal`: number of decimals to include in the rounded number. This is an *optional* argument. If it is not specified, it defaults to 0 and rounds to the nearest integer.
- Returns the rounded number.

Example Usage:

```
# Example 1: Round to hundredth
x = 3.14159
rounded = round(x, 2)
print(rounded) # Prints 3.14

# Example 2: Round to nearest whole number
x = 3.14159
rounded = round(x)
print(rounded) # Prints 3
```

Enumerate

`enumerate(x)` takes an iterable such as a list, dictionary, or string, and adds a counter to the function. It is often used to loop over the indices and values of an iterable simultaneously.

- Accepts two parameters:
  - `x`: an iterable object such as a list, dictionary, or string. This is a *required* parameter.

- `start`: the value to start the counter at. This is an *optional* parameter. If no default value is specified, the counter will start at 0.

- Returns a sequence of numbers coupled with values in given iterable

Example Usage:

```
# Example 1: Iterating over indices and characters in a string
my_string = 'code'
for index, char in enumerate(my_string):
  print(index, char)

# Prints:
# 0 c
# 1 o
# 2 d
# 3 e

# Example 2: Enumerate with start value specified
cereals = ['cheerios', 'fruity pebbles', 'cocoa puffs']
for count, cereal in enumerate(cereals, start=1):
  print(count, cereal)

# Prints:
# 0 cheerios
# 1 fruity pebbles
# 2 cocoa puffs
```

Zip

`zip(x, y)` takes two or more iterables and returns a sequence of tuples where each the first item from each iterable forms the first tuple in the sequence, the second item from each iterable forms a second tuple, and so on. It continues until it has placed all elements in the shortest iterable into a function. Try it

Zip is useful for iterating over multiple iterables in parallel or for combining data from separate iterables.

- Accepts two or more parameters:

  - `x`: an iterable object such as a list, dictionary, or string.

    - `y`: an iterable object such as a list, dictionary, or string.
    - Optionally accepts additional iterables to zip with `x` and `y`
- Returns a sequence of tuples pairing `x[i]` with `y[i]` where `0 <= i <= min(len(x), len(y))`

Example Usage:

```
# Example 1: Zipping Two Lists
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]
zipped = zip(names, ages)
print(list(zipped)) # Prints [('Alice', 25), ('Bob', 30), ('Charlie', 35)]

# Example 2: Zipping Lists of Different Lengths
names = ['Alice', 'Bob', 'Charlie', 'David']
ages = [25, 30, 35]
zipped = zip(names, ages)
print(list(zipped)) # Prints [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
```

Comparing Strings and Lists

Python strings and lists function similarly in many ways, but there are also some major differences between the two data types. Below is a list of the key similarities and differences between strings and lists.

Similarities

- **Ordered sequences** Both strings and lists are ordered sequences of data.
- **Indexed by Integers** Both strings and lists can be indexed using integers (e.g. `lst[0]` or `s[0]`).
- **Slicable** Both strings and lists can be sliced to access a subsection of the string/list (e.g. `lst[1:3]` or `s[1:3]`).
- **Iterable** Both strings and lists are iterable meaning we can loop over them using a for loop.
- **Length** We can use the `len()` function to get the length of either a string or list

Differences

- **Content Type** Both strings and lists are ordered sequences of data, but strings are more limited in the type of data they can contain.
  - Strings are an ordered sequence of character elements.
  - Lists are an ordered sequence of elements that can be of any type, including integers, strings, and other lists.
- **Mutability**
  - Strings are immutable, meaning they are not changeable. To update a string, we must create a new string.

```
s = 'Try'
s[0] = 'C' # Results in TypeError: 'str' object does not support item assignment
```

  - Lists are mutable. We can update the content of a list without creating a new list.

```
lst = ['T', 'r', 'y']
lst[0] = 'C'
print(lst) # Prints ['C', 'r', 'y']
```

Close Section

Bonus Concepts 💡 Not In-Scope

The following syntax is nice to know and may improve your code readability or help you solve certain problems more easily. However, they are not *required* to solve any of the problems in this unit. These are **not in scope for the Unit 3 assessment**, and you do not need to memorize them! Click on the function to read more about how to use it.

- `s.isalpha()` Returns true if all characters in given string are alphabetic letters (a-z).
- `s.isalnum()` Returns true if all characters in given string are alphanumeric (a-z or 0-9).
- `s.find(x)` Returns start index of the first occurrence of substring `x` in a given string. Returns `-1` if `x` is not in the string.

- `s.count(x)` Returns the frequency of the substring `x` in the given string.
- **Sets** A useful data type that allow you to group multiple pieces of data together. Data in sets is unordered, unchangeable, and cannot contain duplicates.

Close Section